

Prep-Center → WMS App

What the warehouse app needs in order to issue a DDT when a driver collects goods

When a driver arrives, the operator scans the boxes leaving and the driver signs on the phone. The app then issues the **DDT** that travels with the goods. Everything for that is already built and tested — the numbering, the document, the signature, the storage. **One endpoint on your side is missing:** given the tracking IDs just scanned, which shipping do they belong to, and what is inside those particular boxes.

The app calls it through the Prep-Center connection already configured in the app's **Settings > API** screen (base URL + key). No new credential is needed.

The call

```
POST {base URL}/shipments/resolve
Authorization: Bearer {the key from Settings > API}
Content-Type: application/json

{ "trackings": ["1Z999AA10123456784", "1Z999AA10123456785"] }
```

Only tracking IDs the app does not already know are sent. A shipping answered once is cached for the rest of the session, so a shipment of 200 boxes costs one call, not 200.

The answer

The shipping and its addresses:

```
{
  "shipments": [
    {
      "shipmentId": "SH-1",
      "fbaId": "FBA15MGXBYBV",
      "amazonReference": "4WNCW3ZN",

      "carrier": "UPS",
      "reason": "Delivery of goods to Amazon FBA on behalf of third party",
      "goodsDescription": "Box",
      "prepAt": "16/09/2026, 13:59",
      "packingGroup": "Pack group 1",
      "legalNote": "The goods are owned by the customer. ...",

      "pickup": { "name": "Simico", "street": "Via Acquasparza 37",
                  "postalCode": "24060", "city": "Grone", "province": "BG",
                  "country": "IT", "phone": "+3903519775012",
                  "email": "info@simico.srl" },
      "customer": { "name": "G&S Commerce GmbH", "city": "Klagenfurt",
                    "country": "AT", "vat": "ATU81467958" },
      "destination": { "name": "Amazon Services FC08/IFC8", "city": "Fiano Romano",
                       "province": "Lazio", "country": "IT" },
      ...
    }
  ]
}
```

...and, in the same object, the boxes themselves:

```

    "boxes": [
      {
        "tracking": "1Z999AA10123456784",
        "weightKg": 18.1,
        "contents": [
          {
            "description": "Corsair MP700 PRO SE 4TB",
            "sku": "AMZIT_1309_B0D7J41STC_DE",
            "asin": "B0D7J41STC",
            "qty": 2 }
        ]
      }
    ]
  }
}

```

<- required: one entry per tracking ID

<- required: what is in THIS box

A tracking ID you do not recognise: simply leave it out. The app flags it to the operator and keeps it off every DDT rather than guessing.

Field names

The adapter already accepts common variants — `shipment_id` / `id` / `shippingId`, `trackingNumber` / `trackingId`, `quantity` / `units`, `zip` / `cap` — and an address may be an object, an array of lines, or a newline-separated string. So the names above are a guide, not a demand. Only **`shipmentId`**, **`boxes[].tracking`** and **`boxes[].contents`** must be present in some form.

Three things that decide whether this works

1. Contents per box — not per shipment

This is the one that matters most, and the one worth pushing back on if it is difficult.

A DDT describes the boxes that actually left. A driver often takes 3 boxes out of a 200-box shipment, and tomorrow's driver takes 4 more — each collection is its own DDT, listing its own goods. Shipment-level or pack-group totals cannot answer *what was in these three boxes*.

If you can only send totals, tell us

A DDT built from pack-group totals would state quantities that never left the building. On a transport document that is a false declaration, not a rounding error — so it is a decision for you, not something the app should paper over.

2. Weight per box

Same reason: the DDT prints the total weight of the boxes collected. If a box's weight is unknown, **omit the field** rather than sending `0` — the app then prints an empty weight instead of “0.00 kg”, which would also be untrue.

3. CORS headers

The app runs inside an Android WebView, so the browser enforces CORS on this call. Without these headers the request never leaves the phone, and it fails silently — nothing reaches your logs:

```

Access-Control-Allow-Origin: *
Access-Control-Allow-Headers: authorization, content-type
Access-Control-Allow-Methods: POST, OPTIONS

```

The OPTIONS preflight must be answered **204** with the same headers and **no auth check** — the preflight itself carries no Authorization header.

If you cannot add these, say so and we move the call to our own backend instead. That also takes the key off the phone, which is the better arrangement anyway; it just needs the credentials to live on our server.

What the app does with the answer

- Groups the scanned boxes by **shipmentId**.
- Issues **one DDT per shipping**, listing only the boxes scanned in that session.
- Sums identical SKUs across those boxes for the products table.
- Adds a **DRIVER SIGNATURE** block — name, company, plate, collection time, and the signature captured on the phone. This is the reason the app generates the document rather than printing yours.
- Numbers it from its own yearly series: DDT 1-OUT/2026, DDT 2-OUT/2026, ...
- Stores your response verbatim beside the document, so what was printed stays reconstructable if a value is ever disputed.